CSE406 Viva Prep — AES + Diffie-Hellman (2105123)

Tumar code er structure dekhe ei question-answer gula banano — ECB/CBC AES, key expansion, GF(2^8) math, DH key exchange, socket-based Alice-Bob communication shob cover kora hoise.

Part 1: AES Fundamentals

Q1. AES ki Feistel cipher? Na, AES হল **Substitution-Permutation Network (SPN)**, Feistel na। Feistel-e proti round e half block operate hoy r baki half unchanged thake, kintu AES-e prottek round e puro state (shobgula byte) simultaneously transform hoy — SubBytes, ShiftRows, MixColumns, AddRoundKey.

Q2. Block size r key size koto? Block size shob shomoy fixed 128 bit (16 byte), তাই code e `word_length=4` (4x4 matrix of bytes = 16 bytes)। Key size variable: 128/192/256 bit — code e eita `initialize_parameter(input_bit)` function e handle kora hoise (`Nk = input_bit // 32`).

Q3. Nr (number of rounds) kivabe calculate hoy? `Nr = Nk + 6`. Tai AES-128 e Nr=10, AES-192 e Nr=12, AES-256 e Nr=14।

Q4. State matrix kivabe fill hoy — row-major na column-major? `convert_plain_bytes_to_state` function dekho — byte gula **column-major order** e fill hoy (`state_matrix[i][j] = plaintext[j*4+i]`), row-major na। Eita standard AES specification onujayi.

Q5. AES round er 4 ta transformation ki ki, r each er purpose ki?

- **SubBytes**: Sbox diye non-linear substitution — confusion আনে, algebraic/linear cryptanalysis thamay
- **ShiftRows**: প্রতি row ke row-index onujayi cyclically left shift kore — column-level diffusion
- **MixColumns**: GF(2^8) e fixed matrix diye column mix kore — deeper diffusion
- **AddRoundKey**: round key er shathe simple XOR — eita e actual key-dependent secrecy আসে

Q6. Last round e MixColumns kno nai? Last round e MixColumns thakle extra computational cost hoy kintu security e kono additional benefit dey na, karon eta pore r kono round nai jekhane etar diffusion propagate hobe। Ei design choice tao AES specification e ensure kore je encryption r decryption structurally symmetric thake (equivalent inverse cipher gothon shohoj hoy)।

Q7. Confusion vs Diffusion (Shannon's principle) — AES e kon kon operation kon kaj kore?

- **Confusion** (ciphertext-key relationship complex kora): SubBytes
- **Diffusion** (ekta plaintext bit change hole onek ciphertext bit change hoye jawa):

ShiftRows + MixColumns AddRoundKey shudhu key mix kore, r structure eishob combine kore full security dey.

Q8. Avalanche effect ki? Plaintext ba key er ekta bit change korle ciphertext er approximately 50% bit change hoye jawa uchit — AES er full-round design eita guarantee kore, ei property ta strong diffusion er result.

Part 2: Key Expansion (Key Schedule)

Q9. Key expansion er `g()` function e ki ki step hoy?

1. **RotWord**: word ta ekta byte cyclically left rotate hoy (code e: `rotated_left_shift`)
2. **SubWord**: প্রতি byte Sbox diye substitute hoy
3. **XOR with Rcon**: প্রথম byte টা `Rcon[round]` er shathe XOR hoy

Q10. Rcon (round constant) er dorkar kno? Rcon na thakle key schedule e ekta symmetry thakto (different rounds e same transformation repeat hoto), jeta slide attack er moto attack ke shohoj kore dito। Rcon round-number-dependent asymmetry add kore.

Q11. Nk > 6 (মানে AES-256) e extra SubWord step kno? Code e dekho: `elif Nk>6 and i%Nk==4: tmp = substitution_only(tmp)`. AES-256 e key onek boro (8 word), tai extra SubWord step add kora hoy jate key material er diffusion aro slow r thorough hoy — extra security margin.

Q12. total_words koyta hoy? `total_words = word_length * (Nr+1) = 4*(Nr+1)`, karon prottek round (0 theke Nr porjonto) er jonno alada 128-bit (4-word) round key lage.

Part 3: GF(2^8) Arithmetic & MixColumns

Q13. AES e GF(2^8) field kno use kora hoy, normal integer arithmetic kno na? Byte-wise operation korte hole eta reversible/invertible hote hobe overflow chara. GF(2^8) e prottek non-zero element er ekta unique multiplicative inverse ache, tai MixColumns er matrix invertible thake — encryption reversible thake without any overflow issue.

Q14. AES er irreducible polynomial (modulus) ki? `x^8 + x^4 + x^3 + x + 1`, hex e `0x11B`। Eta ekta irreducible polynomial GF(2) er upor, jeta GF(2^8) field define korte lage. `aes_helpers.py` e ei modulus ta BitVector diye represent kora hoise (`_AES_MODULUS`).

Q15. Sbox kivabe generate hoy? Duita step: (1) GF(2^8) e byte er multiplicative inverse ber kora (0 er inverse 0), (2) tarpor ekta fixed affine transformation apply kora (bit-shift + XOR combination)। Sbox er non-linearity ei inverse operation theke ashe.

Q16. Tumar code e `CACHE_GF_MULT_TABLE` kno banaisho? Optimization ta explain koro. `mix_column`/`inverse_mix_column` operation e barbar `gf_mult(Mixer[row][k], column[k])` call hoy — kintu `Mixer`/`InvMixer` matrix e mattro koyekta unique value ache (1, 2, 3, 9, 0xB, 0xD, 0xE)। Tai proti unique multiplier er jonno shob 256 possible byte er shathe multiplication result precompute kore rakha hoise ekta dictionary te (`CACHE_GF_MULT_TABLE[multiplier][byte]`)। Eita BitVector-based `gf_mult()` call barbar

korar cost bachay — real BitVector-based `gf_mult` call onek slow (bit-level operation), tai ei caching diye almost ~900+ times speedup paoa gese.

Q17. MixColumns er math ta explain koro. Prottek column ke ekta 4x1 vector dhore, oita ke fixed 4x4 Mixer matrix diye GF(2^8) e matrix-multiply kora hoy:

```
[02 03 01 01]   [s0]
[01 02 03 01] x [s1]
[01 01 02 03]   [s2]
[03 01 01 02]   [s3]
```

Prottek output byte = XOR of GF(2^8) multiplications (karon GF(2^8) e addition = XOR).

Q18. InvMixColumns er matrix ta ki r kno alada? `InvMixer` = Mixer matrix er GF(2^8) inverse matrix (0x0E, 0x0B, 0x0D, 0x09 diye gothito), jate MixColumns tarpor InvMixColumns apply korle original state e fere ashe (Mixer × InvMixer = Identity in GF(2^8))।

Part 4: Modes of Operation (ECB / CBC) & Padding

Q19. ECB mode er weakness ki? Prottek 16-byte block independently encrypt hoy same key diye — tai identical plaintext block always identical ciphertext block dey। Eijonno pattern leak hoy (tumar `2105123_ecb_vs_cbc_image.py` te image encrypt korle eita clearly dekha jay — famous "ECB penguin" problem)।

Q20. CBC mode eita kivabe solve kore? Proti plaintext block encrypt korar age previous ciphertext block er shathe XOR kora hoy (prothom block e random IV use hoy)। Tai identical plaintext blocks o different ciphertext dey, karon chaining er karone context different।

Q21. IV (Initialization Vector) kno lage, r kno eta random/unique hote hoy? IV na thakle first block sensitive — same plaintext+key repeat hole same ciphertext hoye jabe। IV random+unique rakhle prottibar encryption e different ciphertext hoy even same plaintext-key hole o — semantic security ensure hoy। Code e `os.urandom(16)` diye IV generate kora hoise, r IV ta ciphertext er shuru te prepend kora hoy (`ciphertext = list(iv) + ...`)।

Q22. PKCS7 padding kivabe kaj kore? Plaintext ke 16-byte er multiple banate hoy। Koto byte lagbe (`16 - len(data)%16`) shei shonkha diyei pad kora hoy — mane jodi 3 byte lage tahole `[3,3,3]` add hoy। Special case: jodi plaintext already 16 er multiple hoy tao ekta full extra block (16 bytes of value 16) add kora hoy, jate unpadding e ambiguity na thake।

Q23. Unpadding e validation ki ki check kora hoy? Code (`pkcs7_unpadding`) e: pad_value 1-16 range e ache kina, pad_value data er length er cheye choto/shoman kina, r last `pad_value` byte shobgula shei value er shoman kina — na hole "Invalid padding" report kora hoy।

Q24. Padding Oracle Attack ki? Jodi server/receiver "valid padding" vs "invalid padding"

er error message alada bhabe expose kore (timing/response difference), attacker sheita exploit kore byte-by-byte plaintext recover korte pare without knowing the key — eijonno production system e padding error carefully handle kora dorkar (constant-time, generic error)।

Q25. ECB r CBC decryption e error propagation er difference ki?

- **ECB**: ekta ciphertext block e error hole shudhu shei block er decryption nosto hoy
- **CBC**: ekta ciphertext block e bit-error hole shei block toh nosto hoyi, plus **next** block er corresponding bit position eo error propagate hoy (karon oi block XOR hoy previous ciphertext block er shathe)

Part 5: Diffie-Hellman Key Exchange

Q26. DH protocol er mul purpose ki? Ekta insecure/public channel diye duijon (Alice, Bob) shared secret ber kora, jekhane keo age theke kono common secret jane na — eita discrete logarithm problem er hardness er upor security base kora.

Q27. Discrete Log Problem ki? Given `g, p, A = g^a mod p`, `a` ber kora computationally hard (large prime p hole) — eita DH security er foundation।

Q28. Safe prime (`p = 2q+1`, q prime) kno use kora hoy? Safe prime use korle p-1 = 2q er factorization shimito thake (mattro 2 and q) — tai subgroup structure predictable, r small subgroup confinement attack thamano jay। Generator er order hoy either 2, q, ba 2q — `find_generator` function eita check kore (`g^q != 1` and `g^2 != 1` mod p) jate ekta large-order (order = 2q) generator paoa jay, jeta discrete log problem ke hard rakhe (Pohlig-Hellman attack thekay)।

Q29. Miller-Rabin primality test ki, kivabe kaj kore? Probabilistic primality test। `n-1 = 2^d * r` (r odd) e decompose kore, random witness `a` newa hoy, tarpor `a^r mod n` check kora hoy — jodi eita 1 othoba n-1 na hoy, r consecutive squaring o kono step e n-1 na paoa jay, tahole n composite (definite)। Multiple round (tumar code e 40 round) run kore false-positive probability negligible kora hoy (`4^-k` er kache)।

Q30. 40 rounds Miller-Rabin kno enough? Prottek round e false positive probability ⩽ 1/4। 40 round e combined error probability ≈ `4^-40`, jeta practically ignore-able (cryptographically negligible)।

Q31. `generate_private_user_key` e private key kivabe choose kora hoy? Random integer, range `[min(2^(key_size-1), p-2), p-2]` — mane private key ta large enough hote hoy (brute-force resistant) kintu p-2 er beshi na jate valid range e thake।

Q32. Shared secret theke AES key kivabe derive kora hoy? `derive_aes_key(s, key_size_bits)`: shared secret `s` ke `2^key_size_bits` diye mod kore key_size_bits/8 bytes e convert kora hoy (`to_bytes`)। Eta ekta simple truncation-based key derivation — production e proper KDF (jemon HKDF) use kora better hoto, kintu assignment scope e eita shimplified।

Q33. Plain DH er security limitation ki? DH nijei authentication provide kore na — tai **Man-in-the-Middle (MITM) attack** possible: attacker Alice r Bob duijoner shathei alada DH exchange kore, moddhe boshe shob traffic decrypt-reencrypt korte pare, keo bujhte parbe na। Eita thamate hole authenticated DH lagbe (digital signature/certificate diye public value verify kora)।

Q34. `modular_expo` e `pow(base, exp, mod)` use kora hoise — eta efficient kno? Python er built-in `pow()` with 3 arguments **fast modular exponentiation** (square-and-multiply) use kore, complexity O(log(exp)) multiplications — naive repeated multiplication er cheye onek fast, bishesh kore boro exponent (key_size bit) er khetre।

Part 6: Socket Programming / File Transfer

Q35. TCP e message boundary keno thake na, r eita kivabe handle korso? TCP ekta byte-stream protocol — message boundary preserve kore na (ekta `send()` call er data multiple `recv()` e vag hoye jete pare, ba multiple send ekshathe ek recv e ashte pare)। Eijonno `send_message_length_data` e prothome 8-byte length prefix pathano hoy, tarpor actual data — receiving side (`receive_message`) age length porhe, tarpor `receive_exact_message` diye loop kore exactly shei koyti byte na paoa porjonto wait kore।

Q36. `receive_exact_message` e loop kno lage, ekbar `recv()` call korlei to hoy? `recv()` requested amount er cheye kom data return korte pare (partial read, network buffering er karone)। Tai loop kore proti iteration e remaining bytes calculate kore r `recv()` call kora hoy jotokkhon na total_bytes shomogro paoa jay।

Q37. Alice-Bob protocol flow ta summarize koro.

1. Alice: safe prime `p,q` generate kore, generator `g`, private key `k_a`, public value `A = g^k_a mod p` — pathay `(key_size, p, g, A)`
2. Bob: nijer private key `k_b`, `B = g^k_b mod p` compute kore pathay
3. Duijoni nijer private key diye onno jon er public value theke shared secret compute kore (`s_a = B^k_a mod p`, `s_b = A^k_b mod p` — duita shoman hoy)
4. Shared secret theke AES key derive kore
5. Alice CBC mode e encrypt kore filename + ciphertext pathay, Bob decrypt kore.

Q38. Kno CBC use kora hoise ei file/message transfer e, ECB na? File transfer e (bishesh kore image/structured data) ECB er pattern-leakage problem thake, tai CBC use kora hoise jate semantic security thake — same block repeat hole o ciphertext different hoy.

Part 7: Common Tricky / Follow-up Questions

Q39. AES-128, 192, 256 er moddhe security difference ki, r speed tradeoff ki? Beshi key size = beshi rounds (10/12/14) = brute-force theke better protection kintu computation somoy o beshi lage — key schedule r encryption dutoi slow hoy proportionally.

Q40. Tumar code e time measurement (`time.perf_counter()`) kno add korso? Key schedule, encryption, decryption er execution time compare korar jonno — different key

size (128/192/256) er performance impact dekhanor jonno report e.

Q41. `xor_bytes` **r** `xor_matrix` **alada rakhso kno?** `xor_matrix` shudhu 4-byte word er upor kaj kore (key expansion e), `xor_bytes` variable-length byte list er upor kaj kore (CBC chaining e) — different use-case, tai duita function.

Q42. Image encryption e (`2105123_ecb_vs_cbc_image.py`**) tumi ki dekhaisho?** Same image ke ECB r CBC mode e AES diye encrypt kore pixel data compare korso — ECB e image er structure/pattern (edges, shapes) ekhono dekha jay encrypted version e, kintu CBC e completely random-looking noise hoye jay — eita visually prove kore CBC kno beshi secure ECB theke।

Q43. Sbox er ekta important cryptographic property ki? Sbox highly non-linear — eita linear cryptanalysis r differential cryptanalysis er against resistance dey। Plus Sbox invertible (InvSbox exists), jeta decryption possible kore.

Tip: viva te MixColumns er GF(2^8) multiplication hate kore ekta example (jemon `0x02 * 0x63`*) korte bolte pare — mental math e kor।র jonno practice kore rekho (*`aes_helpers.py`*-r bottom e ekta self-test example ache:* `gf_mult(0x57,0x13)=0xFE`*).*